2023 International Conference on Advancement in Computation & Computer Technologies (InCACCT) | 979-8-3503-9648-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/InCACCT57535.2023.10141811

Adam Optimization of Burger's Equation Using Physics-Informed Neural Networks

Soumyendra. Singh
Department of Mathematics, Amrita School of Engineering
Amrita Vishwa Vidyapeetham,
Chennai-601103, India
soumyendrasingh@gmail.com

Dharminder Chaudhary
Department of Computer Science and Engineering, Amrita School of Computing
Amrita Vishwa Vidyapeetham,
Chennai-601103, India
c_dharminder@ch.amrita.edu

B. Yogiraj
Department of Computer Science and Engineering, Amrita School of Computing
Amrita Vishwa Vidyapeetham,
Chennai-601103, India
ch.en.u4aie20005@ch.students.amrita.edu

Ram Narayan Prajapathi
Department of mathematics
Chandigarh University
Chandigarh, India
narain.prajapati@gmail.com

Saurabh Rana
Department of Mathematics (SCSET)
Bennett University
Greater Noida, India
saurabhranapsm@gmail.com

***Abstract*— In this paper, physics informed neural networks are used for numerical approximation of partial differential equations. The data which is used in the process is generated by Latin Hypercube sampling which has been discussed. Adam optimization technique has been implemented to minimize the loss for the discussed partial differential equation. The above proposed methodology has been applied for Burger's equation and the obtained results have been discussed in section 5. Loss function graphs have also been provided to showcase the efficiency of the proposed methodologies.**

Keywords*— Partial Differential Equation; Neural Networks; Adam Optimization; Loss function.

I. INTRODUCTION

A Partial differential equation (PDE) is a function with variables and their derivatives. Using such equations, a function with multiple variables can be related to its partial derivatives. They are critical in studying natural phenomena such as sound, temperature, flow characteristics, and waves. PDEs are used in a variety of fields, including mathematics, engineering and physics[1].

Although PDE's are important in many fields of mathematics, they are notoriously difficult to solve. Simple PDE's can be solved using approximate methods but complex PDE's take millions of CPU hours to solve if they are not impossible to get solved.

In light of recent research and technological advancements, artificial neural networks can approximate partial differential equation solutions much faster than conventional PDE solvers. Once trained, the new neural nets can solve an entire family of PDEs without having to retrain. Neural networks have numerous applications [2, 3, 4, 5, 6] in computer vision, NLP, generative AI and others. These neural networks have now demonstrated their ability to solve PDE's efficiently.

A. Burgers Equation

Burger's equation [7] is one of the most important PDEs with a wide range of applications in science. The one-dimensional Burger's equation has drawn a lot of interest since J.M. Burger's work in the 1940s, largely as an intersection between nonlinear and dissipative events. It has been useful in a wide variety of mathematical problems such as fluid mechanics, hydrodynamics, geometry, and so on.

It is now known that Bateman proposed it for the first time in 1915 after realizing that some viscous flows might be described by its stable solutions. Burgers later suggested it as one of a group of equations providing mathematical representations of turbulence. Due to his numerous contributions and considerable labor, it was given his name.

Burger equation applications include water modelling in unsaturated soil, soil water dynamics, flow statistics, mixing and turbulent diffusion. It is considered as simple version of Navier-Stokes equation because of the non-linear convection term's form and the presence of the viscosity term. Traffic-flow, acoustics, oceanography are some of the use cases which can be solved using this equation [8].

Traditional methods in solving Burger's equation exist such as finite difference method[9]. By traditional methods, it takes a lot of time to solve. Let us see how can we solve it using deep learning techniques.

II. PINN (PHYSICS INFORMED NEURAL NETWORK)

PINNs are neural networks that are designed to learn from data while following the physical laws described by nonlinear PDEs. They are trained to perform supervised learning tasks in this way [10,11]. It is also applied in many machine learning fields [12].

The concept behind PINNs is to encode physical laws and scientific knowledge into learning algorithms in order to

ISBN: 979-8-3503-9648-5/23/$31.00 ©2023 IEEE

improve their robustness and performance. With the addition of this critical information, the algorithms can narrow the field of possible solutions faster and become better at generalizing, which means they can function correctly in the real world with data they have never seen before.

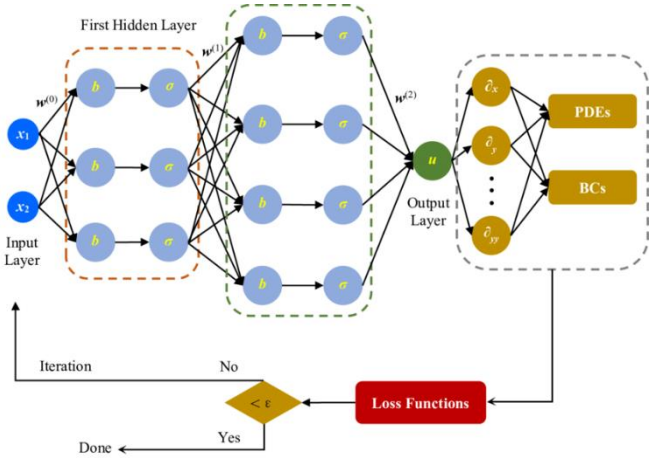

Fig 1. Architecture of PINNs

III. Adam Optimizer

The Adam optimization algorithm combines the principles of momentum optimization and RMSProp. It utilizes an average of previous gradients which decrease exponentially, similar to momentum optimization, and also uses an average of previous squared gradients which are decrease exponentially, similar to RMSProp. Adam stands for adaptive moment estimation [13].

Adam is an optimization algorithm that is applied to optimize an objective function with respect to its parameters. It maintains two vectors, m and v, which are used to store the first and second moments of the gradients, respectively. These vectors are initialized to 0 at the beginning of training. The algorithm also keeps track of a counter, t, which is initialized to 0. At each step of training, the gradients of the objective function with respect to the parameters are computed and stored in the vector g. Adam uses these gradients to update the values of m, v, and t in order to improve the optimization process.

Getting the gradients / derivatives (g) with respect to t.

$$g_t = grad(\theta_{t-1}) \tag{1}$$

Where, θ represents the parameters (weights, biases)
Update the first moment,

$$m_t = \beta_1 m_{t-1} - (1-\beta_1).g_t \tag{2}$$

Where, m_t represents the first moment vector at time t, β_1 represents moving average, generally has a value of 0.9, which means that only last 10 gradients are taken into consideration and previous ones are ignored.
Update the second moment,

$$v_t = \beta_2 v_{t-1} + (1-\beta_2).g_t^2 \tag{3}$$

Where, v_t represents the second moment vector at time t, β_2 also represents the moving average.
Compute the bias-corrected first moment vector,

$$\hat{m}_t = \frac{m_t}{1-\beta_1^T} \tag{4}$$

Compute the bias-corrected second moment vector,

$$\hat{v}_t = \frac{v_t}{1-\beta_2^T} \tag{5}$$

$$\theta_t = \theta_{t-1} + \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \tag{6}$$

Where, α represents the learning rate which decides the step count at each iteration.
Thus, this process continues until the optimal solution is reached by convergence.
Adam includes the hyperparameters,
β_1 is a momentum decay hyperparameter, usually 0.9
β_2 is scaling decay hyperparameter, usually 0.99
ε is smoothing parameter, generally, order of 10^-7
It is proven that Adam optimizer converges better than other convergence algorithms.

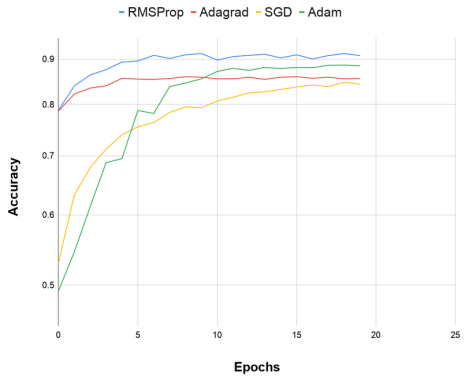

Fig 2. Comparison of convergence of Adam vs Other Optimizers

IV. PINN's for Burger's Equation

The mission of this work is to solve the Burger's equation using PINNs. We can construct the neural network loss function such that, when minimized, the PDE is automatically satisfied. In other words, the most important loss contribution is taken as the residual of the differential equation.

A. Latin Hypercube Sampling

Initially, to create samples of time and location points at first, Latin hypercube sampling is employed. The Latin Hypercube Sampling technique involves splitting your cumulative density function into n equal sections, from which you can sample Latin hypercubes by choosing a random data point from each. Latin Hypercube Sampling is frequently used when training the model to speed up computer processing.

B. Methodology

Using Latin hypercube sampling between (0,1), we are now generating initial and boundary data points for both spatial and temporal values. The initial conditions or function value at t=0 and x=-1 and x=1 are defined as follows.

$$u_t + uu_x - \left(\frac{0.01}{\pi}\right) u_{xx} = 0 \tag{7}$$

where, u represents the latent solution, u_t represents the first derivative of the function u w.r.t time, u_x represents the first derivative of the function u w.r.t position, u_{xx} represents the second derivative of the function u w.r.t position.

With initial conditions,

$x \in [-1, 1]$, $t \in [0,1]$

$u(x,0) = -\sin(\pi x)$

$u(-1,t) = u(1,t) = 0$

The function $f(t,x)$ is defined by:

$$f := u_t + uu_x - \left(\frac{0.01}{\pi}\right) u_{xx} \tag{8}$$

The terms u_t, u_{xx} are calculated using automatic differentiation method [14]. The functions $u(t,x)$ and $f(t,x)$ which are the common factors can be acquired by reducing the mean squared error (MSE).

$$MSE = MSE_u + MSE_f \tag{9}$$

Where,

$$MSE_u = \frac{1}{N_u}\sum_{i=1}^{N_u} |u(t_u^i, x_u^i - u^i)|^2 \tag{10}$$

and

$$MSE_f = \frac{1}{N_f}\sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2 \tag{11}$$

Here $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ represents the starting and boundary data on $u(t,x)$ and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ which depict the collocation points for $f(t,x)$.Equation 10 measures the error between the predicted values and the starting and boundary data, while Equation 11 is the error between the predicted solutions and structure imposed by the given equation for the defined collocation points.

The solution $u(t, x)$ is learnt by the PINN which is constructed using 9 layers with activation function as hyperbolic tangent. The optimization of loss function is done by Adam optimizer.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 32)                96

 dense_1 (Dense)             (None, 32)                1056

 dense_2 (Dense)             (None, 32)                1056

 dense_3 (Dense)             (None, 32)                1056

 dense_4 (Dense)             (None, 32)                1056

 dense_5 (Dense)             (None, 32)                1056

 dense_6 (Dense)             (None, 32)                1056

 dense_7 (Dense)             (None, 32)                1056

 dense_8 (Dense)             (None, 1)                 33

=================================================================
Total params: 7,521
Trainable params: 7,521
Non-trainable params: 0
_________________________________________________________________
```

Fig 3. Structure of the PINN

Finally, the solution $f(t,x)$ is approximated using the latent or hidden solution $u(t,x)$ obtained from the PINN.

V. Results and Discussion

The following are the results of loss values which we have obtained for the different values of starting and boundary data points, Nu and number of collocation points, N_f trained on our PINN, which is made up of 9 layers and *'tanh'* as the activation function.

TABLE I. Loss function values at different collocation points w.r.t number of boundary data points

Nf \ *Nu*	**2000**	**5000**	**7000**	**10000**
100	(5.2) *10^-2	(4.8) *10^-2	(3.9) *10^-2	(4.3) *10^-2
150	(4.1) *10^-2	(4.1) *10^-2	(2.7) *10^-2	(3.9) *10^-2
100	(5.2) *10^-2	(4.8) *10^-2	(3.9) *10^-2	(4.3) *10^-2

We can observe from the table that the prediction loss is getting reduced with the increase in the number of boundary samples or N_u

The predicted function *"u"* with given parameters time *t* and position *x* is almost equal to the function obtained by exact solution.

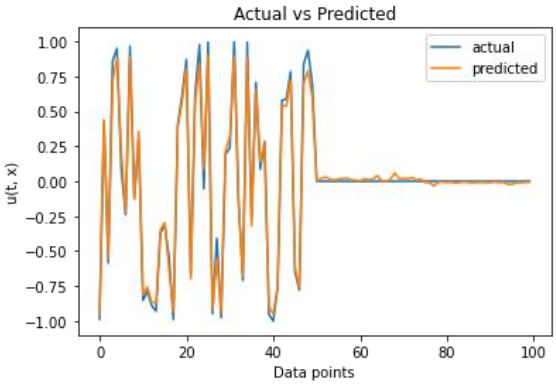

Fig 4. Plot for exact solution versus the predicted solution

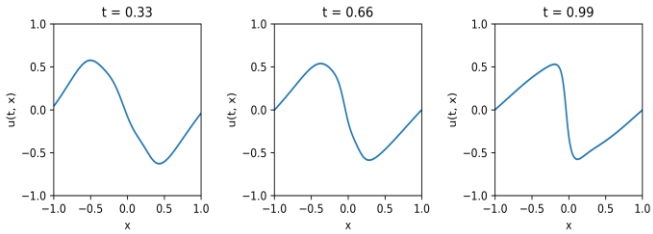

Fig 5. The variation of function "u" at different timestamps.

It shows that our solution is periodical solution.

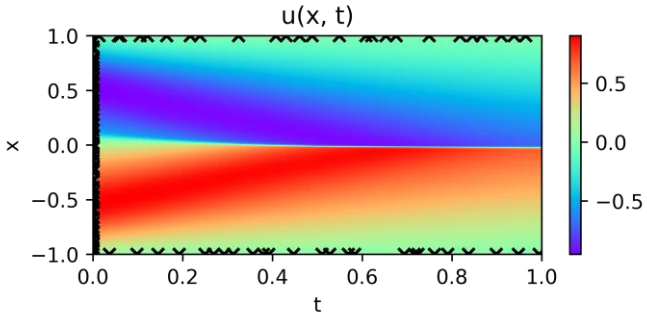

Fig 6. Observed solution with the given initial and boundary data

TABLE II. ACTUAL VS PREDICTED VALUES FOR *NU* BETWEEN 0, 200 AND *NF*=10000

For *Nu* between (0,100)	***Actual - u(t ,x)***	***Predicted – u(t ,x)***
At *Nu* = 50	-0.3363	-0.2879
At *Nu* = 100	0.0000	0.0327
At *Nu* = 150	0.0000	0.0312
At *Nu* = 200	0.0000	-0.0255

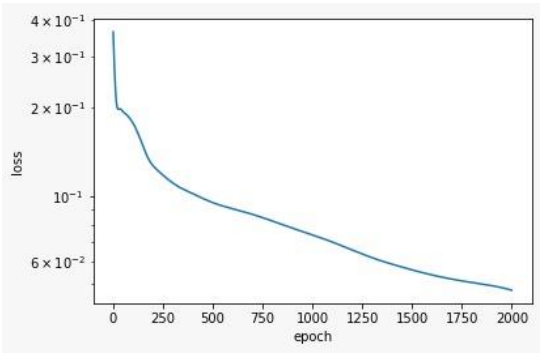

Fig.7(a) Loss curve for *Nu*=100, *Nf* = 5000

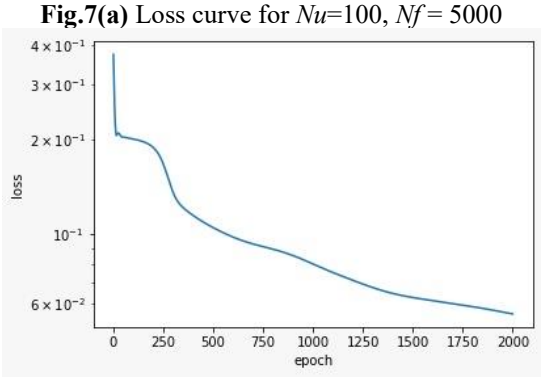

Fig.7(b) Loss curve for *Nu* = 150, *Nf* = 7000

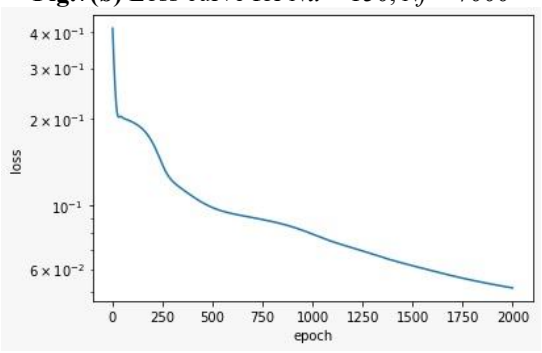

Fig.7(c) Loss curve for Nu=200, Nf=2000

VI. CONCLUSION

Importance of PDE's and their applications are discussed. Burger's equation and its applications are been discussed. Solution to Burger's equation has been implemented using PINNs and the methodology has been explained in detail. A PINN with 9 layers along with *'tanh'* activation function was trained with different number of initial, boundary and collocation points for 2000 epochs. The loss observed was in the order of 10^-2. The loss curves with respect to different number of initial and collocation points have been plotted for 2000 epochs.

REFERENCES

[1] P. K. Jakobsen, "An Introduction to Partial Differential Equations", Arxiv:1901.03022v1, 2019.

[2] R. Anil, K. Manjusha, S. S. Kumar, and K. P. Soman, "Convolutional Neural Networks for the Recognition of Malayalam Characters," in Advances in Intelligent Systems and Computing,

[3] G. M., Vinayakumar Ravi, Sowmya V., E. A. Gopalakrishnan, and S. K. P. "Explainable Deep Learning-Based Approach for Multilabel Classification of Electrocardiogram," in IEEE Transactions on Engineering Management, vol. 68, pp. 1-13, 2021.

[4] A. Krizhevsky, Sutskever, Ilya, Hinton and Geoffrey. "ImageNet Classification with Deep Convolutional Neural Networks". NeurIPS. vol. 25, 2012.

[5] A. Graves, A. -r. Mohamed and G. Hinton, "Speech recognition with deep recurrent neural networks," 2013 IEEE International Conference on Acoustics, Speech and Signal Processing, Vancouver, BC, Canada, 2013, pp. 6645-6649

[6] T. Vignesh, K. V. Kanimozhi, R. Sathish, R. P. Kumar, R. B. Jeyavathana and P. Ezhumalai, "Land Use and Land Cover Classification Using Recurrent Neural Networks with Shared Layered Architecture," 2022 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 2022, pp. 1-10

[7] M. P. Bonkile, et al., "A systematic literature review of Burgers' equation with recent advances," Pramana, vol. 90, pp. 1-21, 2018.

[8] X.-Y. Gao, Y.-J. Guo, and W.-R. Shan, "Taking into consideration an extended coupled (2+1)-dimensional Burgers system in oceanography, acoustics and hydrodynamics," Chaos, Solitons & Fractals, vol. 161, pp. 110870, 2022.

[9] N. A. Mohamed, "Solving one- and two-dimensional unsteady Burgers' equation using fully implicit finite difference schemes," Arab Journal of Basic and Applied Sciences, vol. 26, no. 1, pp. 254-268, Jan. 2019.

[10] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDE," J. Comput. Phys., vol. 378, pp. 686-707, Nov. 2018.

[11] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear PDE," ArXiv:1711.10561, 2017, pp. 10-15.

[12] S. Cuomo and G. F. Di Cola, "Scientific Machine Learning Through Physics-Informed Neural Networks: Where we are and What's Next," J. Sci. Comput., vol. 92, no. 88, 2022.

[13] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ArXiv:1412.6980, 2014.

[14] A. G. B., A. Barak, A. A. R. and J. M. S., "Automatic differentiation in machine learning: a survey," ArXiv:1502.05767, 2015.